

CAMPUS QUEUE

Student frontend presentation preparation

Your speaking notes, packages, files and viva answers

Prepared for project demonstration, code review and academic viva.

8 October 2026

Verified source, practical SQL and honest implementation boundaries. Credentials are deliberately excluded.

Contents

1. Your opening explanation	2
2. How to present your screens	2
3. Packages and tools used	3
4. Important files you should understand	4
5. Components inside App.jsx	5
6. What student-api.js does	5
7. React concepts used in your part	6
8. Retained files and assets	7
9. Complete request and deployment flow	7
10. Viva questions and short answers	7
11. Final rehearsal checklist	8

Your part is the student frontend in `QueueSystemFrontend/`. Prepare in this order: student journey, packages, important files, backend connection and viva questions. This guide reflects the current source on main, checked on 8 October 2026.

The current application defines most screens inside `src/App.jsx`. The separate `src/pages/` and `src/components/` files belong to an earlier implementation and are not imported by the current entry point. Follow the import chain rather than assuming every retained file runs.

1. Your opening explanation

Say: "My part is the student frontend of Campus Queue. It allows students to register, log in, choose a campus service, preview their estimated turn, reserve a place and track their ticket. I used React to build the interface, React Router for navigation, CSS for responsive styling and Fetch to communicate with the Java backend. The frontend displays queue information returned by the backend."

Remember: the frontend displays the position, token and estimate. The backend determines and validates them. Explain your module confidently without claiming ownership of teammates' backend work.

2. How to present your screens

1. Landing page: introduce the application and show signup/login links.

2. Signup or login: explain how the form collects details and sends them to the backend. Use a demonstration account and avoid displaying passwords.
3. Overview: show the available campus services and the student's ticket information.
4. Reservation preview: show projected position, people ahead and estimated service time before confirmation.
5. Confirm reservation: explain that the selected service is sent to the backend, which creates the reservation and returns ticket information.
6. My queue: show token, status, people ahead and counter information. Counter assignment is displayed when available.
7. Bookings: explain Upcoming, Past and Cancelled categories. Demonstrate cancellation with a test booking that is eligible for cancellation.
8. Settings: show the read-only profile, theme switch and logout.

Say "estimated service time", not an exact appointment promise. Reservations are for the same day, without selectable time slots. Original hours are 10 AM-1 PM and 2-3 PM, with lunch from 1-2 PM, Asia/Kolkata. Same-day prebooking is available before opening, subject to backend rules.

3. Packages and tools used

react: builds the interface from components and updates it when state changes.

react-dom: mounts the React application into the HTML page using createRoot.

react-router-dom: handles navigation among overview, queue, bookings and settings. The app uses HashRouter, Routes, Route, Navigate, Link, NavLink, useLocation and useNavigate.

lucide-react: provides icons imported by the current app, including ticket, calendar, theme and logout icons.

gsap: animates panels and route illustrations. The current app respects the reduced-motion preference and reverts the animation context during cleanup.

@phosphor-icons/react: declared as a dependency, but the current App.jsx imports Lucide icons. Do not claim Phosphor powers the current icons.

vite: runs the development server and creates production files in dist/.

@vitejs/plugin-react: adds React support to Vite.

oxlint: checks JavaScript for potential problems through the lint command.

@types/react and @types/react-dom: provide editor/type information and are development dependencies. Their presence does not mean this JSX application is written in TypeScript.

Fetch, URLSearchParams, AbortController and localStorage are browser APIs, not npm packages. The student request helper uses Fetch directly rather than Axios.

Useful commands: npm install installs dependencies; npm run dev starts development; npm run build creates the production bundle; npm run build:tomcat builds and synchronizes files to the Java web application; npm

run lint runs oxlint; npm run preview previews a production build locally.

4. Important files you should understand

All paths below are relative to QueueSystemFrontend/ unless another location is specified.

package.json: lists dependencies and npm commands. Say: "This defines the tools required to run and build the frontend."

package-lock.json: records resolved dependency versions. Say: "It helps team members install a consistent dependency tree."

index.html: contains the root element and loads src/main.jsx. Say: "This is the HTML shell into which React renders the application."

src/main.jsx: imports App and CSS, calls createRoot and renders the application inside StrictMode. Say: "This is the React entry point. StrictMode helps identify development-time issues."

src/App.jsx: contains the current screens, routes, shared interface and theme behavior. Say: "This coordinates the student experience."

src/student-api.js: contains studentRequest and the useStudentBackend custom hook. Say: "This connects the interface to the Java backend and manages server data."

src/queue-layout.js: calculates positions for the queue illustration. Say: "This arranges the visualization; it does not calculate actual queue priority."

src/index.css: provides the main styling, including font definitions. Say: "This establishes the application's base appearance."

src/styles/original-identity.css: applies the current warm visual identity and CSS theme variables. Say: "This customizes the current interface's colours and appearance."

vite.config.js: enables the React plugin and configures the local /api proxy using QUEUE_BACKEND_URL, with a local fallback. Say: "During development, API requests are forwarded to the backend."

netlify.toml: defines the build command, dist publish folder and Node version. The command runs npm run build and the repository's scripts/netlify-redirects.mjs. Say: "This tells Netlify how to build and publish the student website."

scripts/sync-tomcat.mjs: copies built files into the Java web application's ui/ directory and creates student entry pages. It uses Node filesystem and path modules. Say: "This supports serving the frontend through Tomcat locally."

README.md: frontend setup/reference documentation.

DESIGN-NOTES.md: design documentation to help explain visual choices.

Study src/App.jsx and src/student-api.js first. They contain the main interface and data connection.

5. Components inside App.jsx

Brand: displays the Campus Queue logo and home link.

Theme: displays the light/dark appearance switch.

PageHeading: reuses a consistent heading layout across screens.

Ticket: displays service, token, status and counter information. Status text distinguishes waiting, called and serving states.

VirtualQueue: visually represents the student's place in line using queue information and layout coordinates.

Overview: displays the dashboard and service selection.

QueuePage: displays tickets and queue-related actions.

Modal: provides a reusable popup container.

BookingForm: loads the reservation preview and submits the selected service. The reserve button depends on `preview.canReserve`, error state and `backend.busy`.

BookingsPage: filters reservations into Upcoming, Past and Cancelled categories. It uses the India date and booking statuses. It shows a cancellation action for eligible same-day WAITING reservations.

SettingsPage: displays read-only profile details, appearance preference and logout.

LandingPage: provides the public welcome screen.

LoginPage: handles the login/signup interface.

Experience: connects backend state, selected ticket, theme, navigation and screens. It displays loading and error states and chooses the appropriate authenticated or public interface.

App: wraps Experience in HashRouter.

Say: "Reusable components such as Ticket, Modal and PageHeading keep common interface behavior consistent. Experience connects these components to application state and routes."

6. What student-api.js does

studentRequest is the reusable HTTP helper. It builds paths beginning with `/api/student`. It uses GET when there is no payload and POST when a payload is supplied. It sends form data with `URLSearchParams`, includes cookies with credentials: include, reads JSON and throws an error if the HTTP response fails or the backend reports `success: false`. It uses a default 12-second timeout unless the caller supplies an abort signal.

useStudentBackend is a custom React hook. Its state includes profile (logged-in student), ready (initial session check finished), catalog (available services), tickets (queue tickets), bookings (reservations), error (message displayed to the user) and busy (an action is in progress).

refresh loads overview and bookings concurrently through `Promise.all`, then updates React state. A 401 response clears the profile and server-derived lists so the interface no longer treats the student as authenticated.

action performs login, registration, logout and reservation requests. Login/registration set the profile. Logout clears the profile and data. Other successful actions trigger a refresh. A finally block resets the busy state even if a request fails.

On startup, the hook checks /session. It aborts that request when the effect is cleaned up. When a profile exists, it refreshes immediately and approximately every 8 seconds while the page is visible. It also refreshes when the page regains focus or becomes visible. Cleanup removes listeners and clears the interval.

BookingForm separately refreshes the selected service's preview every 5 seconds while open. It prevents overlapping preview requests with an inflight flag and cleans up the timer and abort controller when the selection changes or the form closes.

Say: "I separated backend communication into a custom hook so the UI can use profile, services, tickets and bookings without repeating request logic in each screen."

7. React concepts used in your part

Components: Ticket, Modal and Overview divide the interface into responsibilities.

Props: ticket, profile, catalog and callbacks are passed into child components.

useState: stores theme, selected ticket, selected service, form state and server data.

useEffect: checks sessions, refreshes data, persists the theme and cleans up timers/listeners.

useCallback: keeps the refresh function stable in the backend hook.

useRef: holds a DOM reference used by animations and layout-related work.

useLayoutEffect: available for effects tied to DOM layout before repaint; distinguish it from useEffect, which is normally used for synchronization such as network requests. Read its specific call site in App.jsx before explaining the implementation.

Conditional rendering: shows login, loading, errors or dashboard based on state.

List rendering: map creates navigation items, service options and reservation rows. Keys identify each rendered item.

Controlled inputs: React state controls values such as the selected service.

Routing: Routes selects a screen, Navigate redirects, Link navigates without a full reload, and NavLink can indicate the active route.

Custom hook: useStudentBackend packages reusable state and request behavior.

HashRouter creates URLs such as /#/dashboard. The browser handles the part after #, simplifying route navigation on static hosting.

The theme preference is saved in localStorage and applied using the document's data-theme attribute. Authentication is handled through a backend session, not by treating a theme or other localStorage value as proof of login.

8. Retained files and assets

src/pages/Login.jsx, Dashboard.jsx, Queue.jsx, Bookings.jsx and Settings.jsx: earlier separate screen implementations. Current screens are defined in App.jsx.

src/components/Layout.jsx, Sidebar.jsx and BottomNav.jsx: earlier separate layout/navigation components. Current navigation is implemented inside App.jsx.

src/styles/premium-theme.css: imported by the earlier Layout.jsx, not by the current entry path.

src/App.css: retained stylesheet, not imported by current App.jsx or main.jsx.

public/campus-queue-logo.svg: logo referenced by current Brand.

public/favicon.svg: browser-tab icon referenced by index.html.

public/fonts/: local font assets, including Inter and Outfit, plus licence files. Stylesheets load the relevant fonts.

public/icons.svg and src/assets/hero.png: retained assets. Verify an actual reference before claiming they appear in the current UI.

src/assets/react.svg and vite.svg: retained starter assets.

Say: "The repository retains earlier implementations. The active application starts at main.jsx and imports App.jsx, so I identify current code by following that import chain."

9. Complete request and deployment flow

Student clicks Reserve my place -> BookingForm submits the selected service -> backend.action calls studentRequest -> POST /api/student/bookings/create -> Java validates and saves the reservation -> frontend refreshes overview and bookings -> React displays the updated ticket.

The booking preview is fetched before confirmation. It informs the student about projected position, people ahead, estimated time and any warning. The backend must revalidate the actual reservation because other students can book after a preview is displayed.

Netlify hosts the student frontend at <https://campusqueue.netlify.app/>. Its /api requests are proxied to the Java backend at <https://queuesystem-1.onrender.com>. The backend accesses MySQL on Railway. Database credentials stay on the backend and must not be placed in frontend code or this study guide.

Say: "Netlify hosts the student frontend. Requests to /api are proxied to the Java backend on Render, which accesses MySQL on Railway. Database credentials stay on the backend."

The cloud build publishes dist/. The local Vite proxy is a development configuration; the Netlify redirect generation handles cloud routing. scripts/sync-tomcat.mjs supports a separate local Tomcat serving arrangement.

10. Viva questions and short answers

Q: Why React?

A: It supports reusable components and updates the interface when application state changes.

Q: Is queue tracking real-time?

A: It uses periodic polling, roughly every 8 seconds while visible, rather than WebSockets. The preview polls every 5 seconds while open.

Q: Where is login stored?

A: Authentication uses a backend session and cookies. Local storage is used for the theme preference.

Q: Does the frontend calculate the token or ETA?

A: No. It displays backend values. queue-layout.js only calculates illustration positions.

Q: What happens if the backend fails?

A: The request helper raises an error and the interface displays a message with retry behavior.

Q: Can a student edit the profile here?

A: Current settings fields are read-only.

Q: How do you avoid repeated submissions?

A: Busy state disables relevant buttons while an action runs. The backend must also enforce reservation rules.

Q: What is the difference between development and production?

A: Vite serves the application during development. The build command generates production files in dist/.

Q: Why separate API logic?

A: A shared request helper and custom hook reduce duplication and centralize session handling, refresh logic and errors.

Q: Why clean up effects?

A: Clearing timers, removing listeners and aborting requests prevents stale updates and repeated background work after a component unmounts or dependencies change.

Q: Is the estimated service time guaranteed?

A: No. It is a backend estimate that can change as the queue and service progress change.

Q: Can hiding a button enforce security?

A: No. The frontend guides the interaction; the backend must enforce authentication, ownership and reservation rules.

11. Final rehearsal checklist

1. Practise the opening explanation without reading it.
2. Walk through landing, login, overview, reservation preview, ticket, bookings and settings.
3. Open main.jsx, App.jsx and student-api.js and explain how they connect.

4. Explain props, state, effects, the custom hook, routing and polling with examples from these files.
5. Distinguish the queue illustration from the backend's actual ordering and ETA calculation.
6. Explain current files versus retained older files.
7. Rehearse with a demonstration account during operating hours when showing staff-driven queue progression. Allow extra time for the free backend to wake up.
8. Keep the presentation runbook and master guide available for deeper backend/database questions. Do not promise untested behavior.